

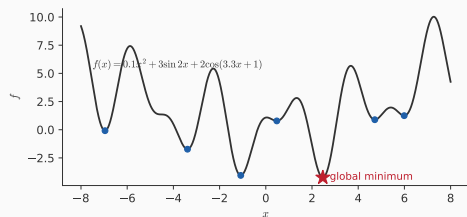
# Optimization

## 5.2 Simulated annealing

---

José Firmino Aguilar Madeira

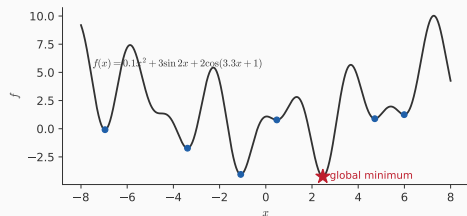
# From GLODS to SA: what if we have only one point?



The chapter's reference function. From  $x_0 = 6$ , a local method (Chs. 2–4) ends at  $x \approx 5.99$  ( $f \approx 1.25$ ), not at  $x^* = 2.47$ : to get there one must **climb** two barriers (the higher one at  $x \approx 3.68$ ).

**5.1 (GLODS):** several searches, to find other basins.

# From GLODS to SA: what if we have only one point?

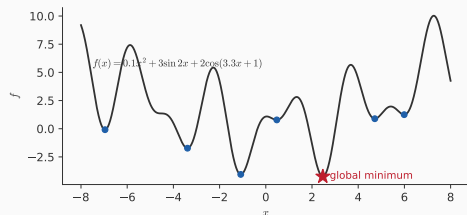


The chapter's reference function. From  $x_0 = 6$ , a local method (Chs. 2–4) ends at  $x \approx 5.99$  ( $f \approx 1.25$ ), not at  $x^* = 2.47$ : to get there one must **climb** two barriers (the higher one at  $x \approx 3.68$ ).

**5.1 (GLODS):** several searches, to find other basins.

**Question:** what if we keep **only one point**, but allow it to **cross barriers**?

# From GLODS to SA: what if we have only one point?



The chapter's reference function. From  $x_0 = 6$ , a local method (Chs. 2–4) ends at  $x \approx 5.99$  ( $f \approx 1.25$ ), not at  $x^* = 2.47$ : to get there one must **climb** two barriers (the higher one at  $x \approx 3.68$ ).

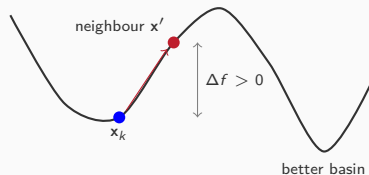
**5.1 (GLODS):** several searches, to find other basins.

**Question:** what if we keep **only one point**, but allow it to **cross barriers**?

## The idea of simulated annealing

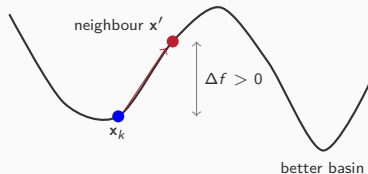
Leaving a local minimum requires **sometimes accepting uphill moves**.

# Accept an uphill move? The Metropolis criterion



**Question:** the neighbour is worse. Do we accept it?

# Accept an uphill move? The Metropolis criterion

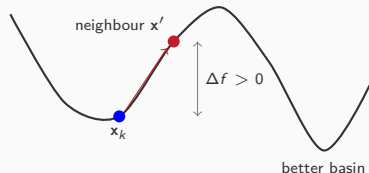


**Question:** the neighbour is worse. Do we accept it?

A local method:  $\Delta f > 0 \Rightarrow$  **no**. Simulated annealing:

$\Delta f > 0 \Rightarrow$  **maybe**.

# Accept an uphill move? The Metropolis criterion

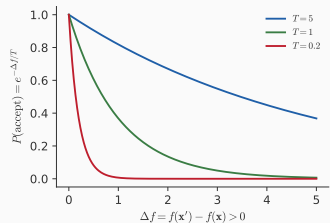


**Question:** the neighbour is worse. Do we accept it?

A local method:  $\Delta f > 0 \Rightarrow$  **no**. Simulated annealing:  $\Delta f > 0 \Rightarrow$  **maybe**.

## Metropolis criterion

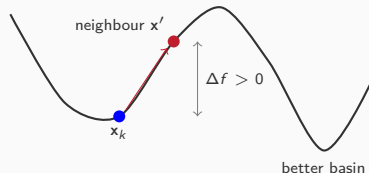
Always accept if  $\Delta f = f(x') - f(x_k) \leq 0$ ; if  $\Delta f > 0$ , accept with probability  $P(\text{accept}) = e^{-\Delta f/T}$ .



The same uphill move  $\Delta f = 1$ : accepted with prob. 0.82 at  $T = 5$ , 0.37 at  $T = 1$ , 0.007 at  $T = 0.2$ .

**$T$  is not a physical temperature:** it is a parameter on the scale of  $f$ .

# Accept an uphill move? The Metropolis criterion

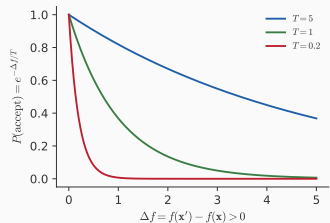


**Question:** the neighbour is worse. Do we accept it?

A local method:  $\Delta f > 0 \Rightarrow$  **no**. Simulated annealing:  $\Delta f > 0 \Rightarrow$  **maybe**.

## Metropolis criterion

Always accept if  $\Delta f = f(x') - f(x_k) \leq 0$ ; if  $\Delta f > 0$ , accept with probability  $P(\text{accept}) = e^{-\Delta f/T}$ .



The same uphill move  $\Delta f = 1$ : accepted with prob. 0.82 at  $T = 5$ , 0.37 at  $T = 1$ , 0.007 at  $T = 0.2$ .

**$T$  is not a physical temperature:** it is a parameter on the scale of  $f$ .

Where the name comes from: a heated metal that is **cooled slowly** crystallizes in a minimum-energy state; cooled quickly, it ends up in a higher-energy state. Kirkpatrick, Gelatt and Vecchi (1983) carried the idea over to optimization with the Metropolis criterion (1953).



# Hot → cold: exploration → refinement

## High $T$

$P(\text{accept uphill}) \approx 1$ : wanders between basins — **exploration**.



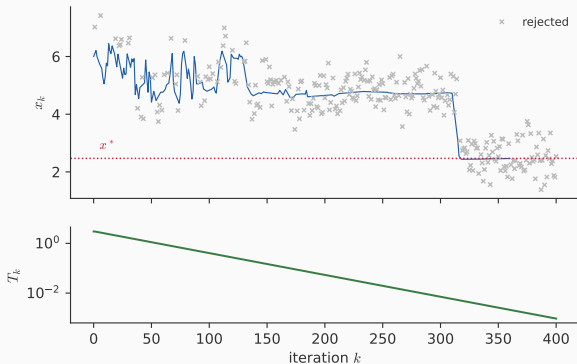
## Intermediate $T$

accepts some uphill moves — **transition**: settles in a basin, with excursions.



## Low $T$

$P(\text{accept uphill}) \approx 0$ : only descends — **refinement**.



One run on the reference function: accepted (blue) and rejected (grey); below,  $T_k$  on a log scale.

exploration → refinement — the language of the chapter opening, in a single decreasing parameter.

## Simulated annealing: $\mathbf{x}_0$ , $T_0$ , $c \in (0, 1)$ , step

$\sigma$ ,  $N_{\max}$

- 1: set  $\mathbf{x} = \mathbf{x}_0$ ;  $f_{\mathbf{x}} = f(\mathbf{x})$ ;  $\mathbf{x}_{\text{best}} = \mathbf{x}$ ;  $f_{\text{best}} = f_{\mathbf{x}}$ ;  $T = T_0$
- 2: **for**  $k = 1, \dots, N_{\max}$  **do**
- 3:   generate  $\mathbf{x}' = \mathbf{x} + \sigma \zeta$ ,  $\zeta \sim N(\mathbf{0}, I)$ , projected onto  $X$   $\triangleright$   
    random neighbour
- 4:   compute  $f' = f(\mathbf{x}')$ ;  $\Delta f = f' - f_{\mathbf{x}}$   $\triangleright$  the iteration's only  
    evaluation
- 5:   **if**  $\Delta f \leq 0$  **or**  $\xi < e^{-\Delta f/T}$ ,  $\xi \sim U(0, 1)$  **then** set  $\mathbf{x} = \mathbf{x}'$ ;  
     $f_{\mathbf{x}} = f'$
- 6:   **end if**
- 7:   **if**  $f_{\mathbf{x}} < f_{\text{best}}$  **then** set  $\mathbf{x}_{\text{best}} = \mathbf{x}$ ;  $f_{\text{best}} = f_{\mathbf{x}}$
- 8:   **end if**
- 9:   set  $T = c T$   $\triangleright$  geometric cooling
- 10: **end for**
- 11: **return**  $\mathbf{x}_{\text{best}}$

$\mathbf{x}_{\text{best}} \neq \mathbf{x}_k$  in general

We return the **best visited**: the current point may have got worse *on purpose* (in the earlier descent methods — line search, direct search —  $f$  at the iterate never increases). One evaluation of  $f$  per iteration,  $n_f = N_{\max} + 1$ ; only values of  $f$ .

## Discrete variables

$x_i \in \{1, \dots, 5\}$  (thickness, type, component): change a random variable to the adjacent category,  $(2, 4, 1, 3) \rightarrow (2, 4, 2, 3)$ . Permutations: swap two positions,  $(1, 2, 3, 4, 5) \rightarrow (1, 4, 3, 2, 5)$ . **No continuous distance and no derivatives: it is enough to know how to generate a neighbour.**

# How fast to cool? Practice and theory

## Too fast

$T$  drops before the point has explored: it quickly behaves like a local method and stays in the first basin.

## Too slow

Explores better, but costs many evaluations; with a fixed  $N_{\max}$  it may never get to refine.

Usual schedules: geometric  $T_{k+1} = c T_k$  ( $c \approx 0.9$ – $0.999$ ), or stepwise ( $n_T$  iterations at each  $T$ ).

Geman, S., Geman, D., IEEE Trans. Pattern Anal. Mach. Intell. 6(6):721–741, 1984.

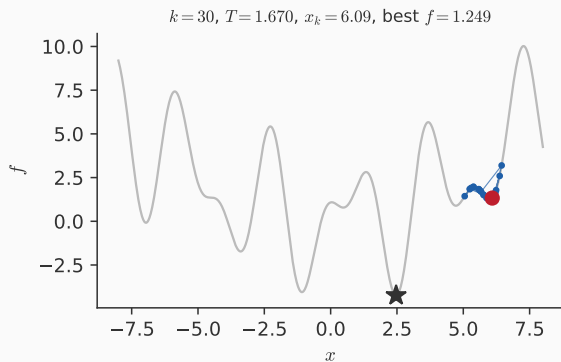
Hajek, B., *Cooling schedules for optimal annealing*, Math. Oper. Res. 13(2):311–329, 1988.

## Theory (Geman & Geman, 1984; Hajek, 1988)

In a finite space, with logarithmic cooling  $T_k = d / \ln(1 + k)$ ,  $k \geq 1$ , and  $d \geq d^*$  (depth of the deepest local, non-global minimum), the probability of being at a global optimum tends to 1.

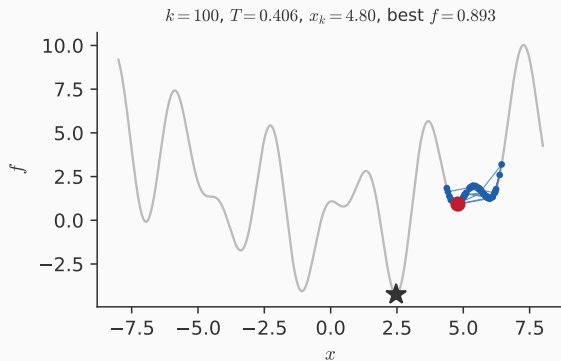
The theoretical guarantee requires cooling too slow to be normally used in practice. Practical (geometric) schedules do not have this guarantee (the best visited point has at least the guarantee of random search, 3.2.1 — equally slow).

## One run step by step: $x_0 = 6$ , $T_0 = 3$ , $c = 0.98$ , $\sigma = 0.5$ , $N_{\max} = 400$



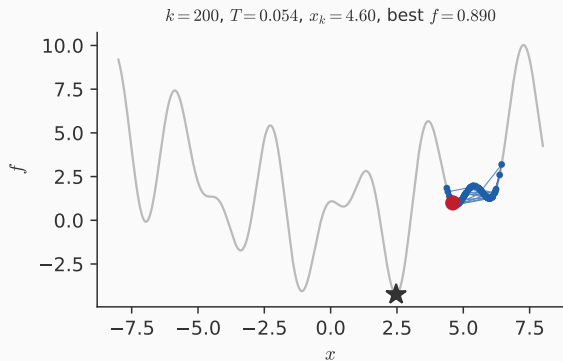
- $k = 30$ ,  $T \approx 1.7$ : still hot; goes up and down with a high acceptance rate around  $x_0$ .

## One run step by step: $x_0 = 6$ , $T_0 = 3$ , $c = 0.98$ , $\sigma = 0.5$ , $N_{\max} = 400$



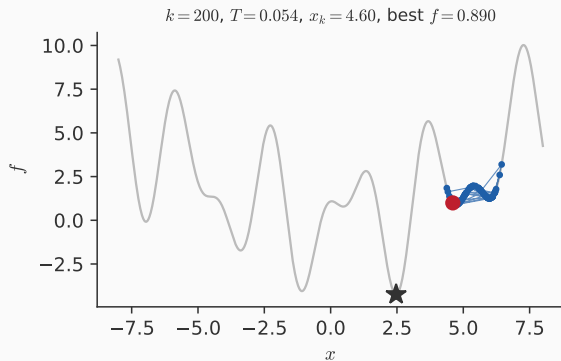
- $k = 30$ ,  $T \approx 1.7$ : still hot; goes up and down with a high acceptance rate around  $x_0$ .
- $k = 100$ ,  $T = 0.4$ : has settled in the basin of  $x = 4.7$  (local minimum,  $f = 0.89$ ), with excursions.

## One run step by step: $x_0 = 6$ , $T_0 = 3$ , $c = 0.98$ , $\sigma = 0.5$ , $N_{\max} = 400$



- $k = 30$ ,  $T \approx 1.7$ : still hot; goes up and down with a high acceptance rate around  $x_0$ .
- $k = 100$ ,  $T = 0.4$ : has settled in the basin of  $x = 4.7$  (local minimum,  $f = 0.89$ ), with excursions.
- $k = 200$ ,  $T = 0.05$ : still in the basin of 4.7; excursions are now rare and short.

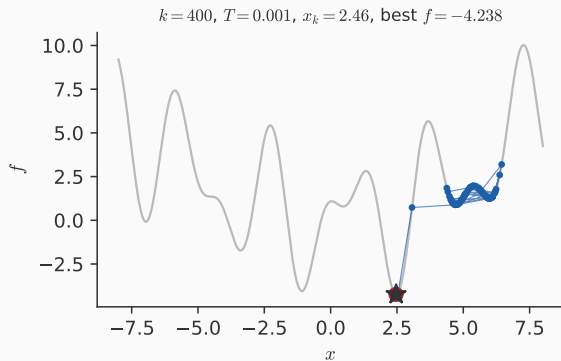
## One run step by step: $x_0 = 6$ , $T_0 = 3$ , $c = 0.98$ , $\sigma = 0.5$ , $N_{\max} = 400$



- $k = 30$ ,  $T \approx 1.7$ : still hot; goes up and down with a high acceptance rate around  $x_0$ .
- $k = 100$ ,  $T = 0.4$ : has settled in the basin of  $x = 4.7$  (local minimum,  $f = 0.89$ ), with excursions.
- $k = 200$ ,  $T = 0.05$ : still in the basin of 4.7; excursions are now rare and short.

**Question:** do you think it will manage to escape? (*It is stochastic: it may or may not.*)

## One run step by step: $x_0 = 6$ , $T_0 = 3$ , $c = 0.98$ , $\sigma = 0.5$ , $N_{\max} = 400$



- $k = 30$ ,  $T \approx 1.7$ : still hot; goes up and down with a high acceptance rate around  $x_0$ .
- $k = 100$ ,  $T = 0.4$ : has settled in the basin of  $x = 4.7$  (local minimum,  $f = 0.89$ ), with excursions.
- $k = 200$ ,  $T = 0.05$ : still in the basin of 4.7; excursions are now rare and short.

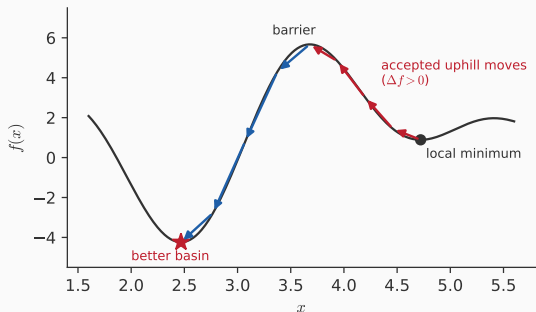
**Question:** do you think it will manage to escape? (*It is stochastic: it may or may not.*)

At  $k = 315$  a proposal lands **on the other side of the barrier**, at a lower point, and is accepted:  $x = 2.464$ , in the basin of the global optimum ( $x^* = 2.468$ ).

A single run (seed 3 of the Python figure script),  $n_f = 401$ . **It went well this time.**



# How is a barrier crossed?



From the basin of 4.72 to that of 2.47: barrier of height  $H \approx 4.8$ .

SA does not know where the better basin is. There are two ways to get there:

## A sequence of accepted climbs

At roughly constant  $T$ , uphill moves with total rise  $H$  have an acceptance factor  $e^{-H/T}$ : 0.20 at  $T = 3$ , 0.008 at  $T = 1$ ,  $6 \times 10^{-6}$  at  $T = 0.4$ .

Low  $T$  traps.

## A jump

A proposal lands directly on the other side, at a lower point — accepted as a descent. This is what happened in the previous run ( $k = 315$ : from 4.73 to 3.07, at  $3.3\sigma$ ). A small  $\sigma$  makes such jumps rare.

More budget: more attempts.

## One run is not enough: 100 runs

Success ( $|x - x^*| < 0.3$ ) in 100 runs, always from  $x_0 = 6$ :

| $\sigma$ | $T_0$ | $c$   | $N_{\max}$ | $T_{N_{\max}}$        | success |
|----------|-------|-------|------------|-----------------------|---------|
| 0.5      | 3.0   | 0.980 | 400        | $9.3 \times 10^{-4}$  | 35%     |
| 0.5      | 3.0   | 0.900 | 400        | $1.5 \times 10^{-18}$ | 19%     |
| 0.5      | 3.0   | 0.995 | 400        | $4.0 \times 10^{-1}$  | 74%     |
| 0.5      | 0.3   | 0.980 | 400        | $9.3 \times 10^{-5}$  | 20%     |
| 1.0      | 3.0   | 0.980 | 400        | $9.3 \times 10^{-4}$  | 88%     |
| 0.5      | 3.0   | 0.980 | 2000       | $8.5 \times 10^{-18}$ | 73%     |

With the parameters of the previous run, only **35%**. The successful run was one among many possible: **one SA run is not an experimental conclusion.**

**Stochastic algorithm  $\Rightarrow$  several runs**

Report the **success rate**, **median** and **spread** over  $\geq 20$ –30 runs, with fixed seeds.

The same holds for genetic algorithms (5.3).

# What do the parameters do?

| parameter           | too small                                | too large                                              |
|---------------------|------------------------------------------|--------------------------------------------------------|
| $T_0$               | starts almost local ( $T_0 = 0.3$ : 20%) | accepts almost everything; wastes iterations wandering |
| $\sigma$            | does not cross barriers                  | proposals too far away, few accepted                   |
| $c$                 | cools quickly ( $c = 0.9$ : 19%)         | long exploration; may never get to refine              |
| $N_{\max}$ (budget) | insufficient budget                      | higher cost ( $N_{\max} = 2000$ : 73%)                 |

## $T$ : what we accept

tolerance to getting worse

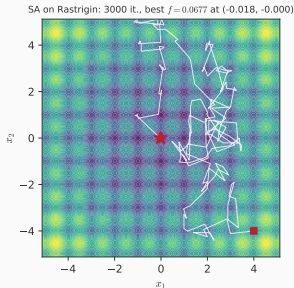
## $\sigma$ : where we propose

geometric exploration — matched to the scale of the basins (here,  $\sigma = 1$ : 88%)

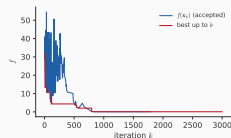
$T$  and  $\sigma$  play different roles; the cooling schedule ( $T_0$ ,  $c$ ,  $N_{\max}$ ) balances exploration and refinement.

## In two dimensions: Rastrigin

$f(\mathbf{x}) = 10n + \sum_i (x_i^2 - 10 \cos 2\pi x_i)$ ,  $n = 2$ , on  $[-5.12, 5.12]^2$ : 121 local minima on a grid; global  $f = 0$  at  $\mathbf{0}$ .



On this problem, even without accepting uphill moves ( $T \approx 0$ ), it gives 97% (30 runs); the effect of cooling shows on the 1D function (19% with  $c = 0.9$ ).



- $T_0 = 20$ ,  $c = 0.995$ ,  $\sigma = 0.5$ ,  $N_{\max} = 3000$ , from  $(4, -4)$ :  $f = 0.068$ ,  $|\mathbf{x}| \approx 0.02$  (global basin). First a walk over the grid of minima; once  $T \lesssim 1$  it settles.
- 30 runs from random points: success ( $f < 1$ ) 100% with  $c = 0.995$ ; 97% with 0.9–0.99; 90% with 0.999 ( $T_{3000} \approx 1$ : it never cooled down). On another sample of 30 points (the one in 5.3): Nelder–Mead **7%** ( $n_f \approx 87$ ), SA 100% ( $n_f = 3001$ ).
- Modest final accuracy ( $f \sim 10^{-2}$ ): finish with a local descent (Ch. 3) from  $\mathbf{x}_{\text{best}}$ .

# The price of simulated annealing

## Advantages

- Extremely simple.
- Only values of  $f$ .
- Can escape from local minima.
- Adapts to discrete variables and permutations through the neighbourhood.
- One evaluation per iteration.

## SA → local method

SA finds a good basin; a local method (Chs. 2–3) refines. In the example: from  $x_{\text{SA}} = 2.464$ , a direct search reaches  $|x - x^*| < 10^{-6}$  with **37 evaluations** (SA used 401).

## Limitations

- Many iterations.
- Random result.
- $T_0$ ,  $c$ ,  $\sigma$  strongly affect performance.
- More effective at exploring than at refining to high accuracy.
- The global guarantee is asymptotic, with impractically slow schedules.

Exploring globally and refining locally are different tasks — and can be done by different methods (the GLODS philosophy, seen another way).

SA does not “know” it has found the global one: it returns the best visited point, with no certificate of globality.

## GLODS and SA: what changed?

|                | GLODS (5.1)                           | SA (5.2)                     |
|----------------|---------------------------------------|------------------------------|
| State          | several points                        | one point                    |
| How it escapes | the <i>search</i> finds other regions | accepts uphill moves         |
| Local search   | direct search ( <i>poll</i> )         | not explicit (random walk)   |
| Memory         | list of minima / searches             | current point + best visited |
| Random?        | depends on the <i>search</i>          | yes                          |
| Natural output | several minima                        | the best visited             |

It is not “which is better”: they are two different answers to the same question of the chapter.

# On the computer: MATLAB implementation

```
% core of SimulatedAnnealing(f, x0, T0, c, sigma, Nmax, lb, ub)
for k = 1:Nmax
    xn = x + sigma*randn(size(x));           % neighbour
    xn = min(max(xn, lb), ub);               % project onto X
    fn = f(xn); df = fn - fx;
    if df <= 0 || rand < exp(-df/T)           % Metropolis
        x = xn; fx = fn;
        if fx < fbest, xbest = x; fbest = fx; end
    end
    T = c*T;                                % cool
end
```

The full function (SimulatedAnnealing, with info.nfev and info.nacc) is in the course companion code.

```
f = @(x) 20 + sum(x.^2 - 10*cos(2*pi*x));
for s = 1:30                                % 30 runs
    rng(s); x0 = -5.12 + 10.24*rand(1,2);
    [x, fx, info] = SimulatedAnnealing(f, x0, ...
        20, 0.995, 0.5, 3000, -5.12, 5.12);
    fs(s) = fx;                             % info.nfev = 3001
end
median(fs), prctile(fs, [25 75]), mean(fs < 1)
```

rng(s) makes each run reproducible; the result is reported with median, quartiles and success rate. MATLAB has simulannealbnd (Global Optimization Toolbox), with adaptive cooling and step size, and reannealing. Projecting onto the bounds (min/max) changes the distribution of neighbours near the boundary: it is just a simple option.

## KEY TAKEAWAYS — 5.2

1. **Global exploration:** a **single point** that occasionally accepts getting worse, to cross barriers.
2.  $T$  controls the **acceptance** of uphill moves (high: exploration; low: refinement);  $\sigma$ , the **neighbourhood**.
3. **Information and cost:** only values of  $f$ ; one evaluation per iteration, many iterations.
4. **Randomness:** one run is not enough — several runs; success rate, median and spread.
5. **Guarantee:** current point: only asymptotic (finite space, logarithmic cooling — impractical); best visited: that of random search (3.2.1), equally slow. With a finite budget, **no certificate of global optimality**.

**GLODS:** several points exploring different basins. **SA:** one point that can move uphill.

What if we explored several regions simultaneously and let the good solutions exchange information?

→ population → 5.3 Genetic algorithms.



# References and resources

**References** Full list: *Course bibliography*, at the end of the course PDF.

- Kirkpatrick, S., Gelatt Jr., C. D., Vecchi, M. P., *Optimization by simulated annealing*, Science 220(4598):671–680, 1983.
- Hajek, B., *Cooling schedules for optimal annealing*, Mathematics of Operations Research 13(2):311–329, 1988.
- Hillier, F. S., Lieberman, G. J., *Introduction to Operations Research*, 2024 Release, McGraw-Hill, 2024 — metaheuristics.

## Resources

- Companion code: `SimulatedAnnealing` (`info.nfev`, `info.nacc`). MATLAB: `rng(s)`; `simulannealbnd` (Global Optimization Toolbox).
- Student projects (internships at Tecmic, real data): Santos, J. R. (2022) and Costa, M. O. (2023), allocation of transport services; Jin, C. (2024), vehicle assignment for Carris, with SA and NSGA-II; Macau, R. O. (2026), driver scheduling, comparing SA, tabu search and greedy. Full references in the *Course bibliography* (reports in Portuguese).